Jurnal Teknik Informatika (JUTIF)
P-ISSN: 2723-3863
E-ISSN: 2723-3871
Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

Development of a Telegram-Based Automated Penetration Testing System Based on PTES and OWASP WSTG Version 4.2

Ahmad Izza Abdillah[1], Ira Rosianal Hikmah[*2], Rahmat Purwoko[3], Arizal[4]

[1,2,3,4]Cyber Security Engineering, National Cyber and Crypto Polytechnic, Indonesia

Email: [2]ira.rosianal@poltekssn.ac.id

Received : Feb 10, 2026; Revised : Apr 1, 2026; Accepted : Apr 8, 2026; Published : Oct 10, 2026

Abstract

The security of Electronic-Based Government Systems (SPBE) is imperative for protecting public service websites and safeguarding sensitive data. Persistent security threats, such as web defacement incidents, necessitate the adoption of more efficient and accessible security assessment methodologies. This research addresses this challenge by proposing an automated penetration testing tool integrated into a Telegram bot interface, developed using the Software Development Life Cycle (SDLC) methodology. This study contributes a fully integrated, Telegram-driven penetration testing framework aligned with standardized security testing protocols, specifically the Penetration Testing Execution Standard (PTES) and OWASP WSTG v4.2. A key technical advancement is the integration of proxychains technology to prevent IP address blocking during automated scans. Evaluation through User Acceptance Testing (UAT) yielded an overall score of 95.75%, confirming its excellent reliability (96%), usability (96%), and efficiency (95.2%). Performance testing validated the tool's capability to detect four out of five predefined vulnerability types. This research concludes that the tool is a viable solution for enhancing the security posture of government websites through standardized and regular security audits.

Keywords : *Automated Penetration Testing Tool, OWASP WSTG v4.2, PTES, SDLC, Telegram Bot.*

This work is an open access article and licensed under a Creative Commons Attribution-Non Commercial 4.0 International License

1. INTRODUCTION

In the continuously evolving digital era, cybersecurity has become a critical aspect for government institutions that manage sensitive data and provide public services through electronic systems [1]. According to data from the Ministry of Administrative and Bureaucratic Reform (PANRB), Indonesia's Electronic-Based Government System (SPBE) index improved in 2023, achieving a "Good" category with a score of 2.79 [2]. This improvement indicates the implementation of SPBE standards across various agencies. As a manifestation of SPBE implementation, government websites play a vital role in disseminating information and interacting with the public. Applying robust cybersecurity to these sites is essential for maintaining public trust and ensuring the continuity of government services [3]. Amidst the improvement in the SPBE index, cybersecurity threats remain a serious concern. According to the 2024 Indonesian Cybersecurity Landscape, the five most common types of cyber incident response assistance are data breaches, illegal access, web defacement, ransomware, and distributed denial of service (DDoS). A total of 4,071 online gambling web defacements targeting government websites were detected using the dorking method from April to December, with the highest number of cases occurring in May [4].

The phenomenon of online gambling content infiltration on government websites in Indonesia has become a serious issue in recent years. The Ministry of Communication and Information Technology reported handling 23,616 instances of online gambling page insertions on government sites and 22,205 on educational institution sites between 2023 and 2024 [5]. At the regional level, researchers conducted

Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

an interview with a local government agency, which revealed that in 2024, incidents of online gambling content injection occurred at an average frequency of 10 times per month, indicating vulnerabilities in local government websites. The agency addressed these issues through manual penetration testing, following the stages of enumeration, scanning, exploitation, and reporting. Recent research by Zagi et al. in 2025 found that 346 web pages were defaced within one month, including 115 .go.id domains, using infiltration techniques such as hidden URL injection, redirection, and keyword stuffing with popular terms like "slot," "togel," and "gacor." The average remediation time for affected sites reached approximately 75 hours [6]. Based on these findings, the injection of online gambling content on government websites in Indonesia is identified as a recurring attack pattern. In accordance with BSSN Regulation No. 4 of 2021, Electronic-Based Government System (SPBE) applications are required to undergo regular security testing, including penetration testing, to ensure protection against cyber threats [7]. Aligned with this regulation, a more efficient approach to penetration testing is needed. Manual methods are deemed inadequate as they require significant time and financial resources [8].

Several studies have explored the automation of penetration testing using Telegram bots. Hafiz et al. developed a Telegram bot to remotely execute seven penetration testing tools on a Kali Linux server. However, their research still required users to have technical knowledge of the tools and was not yet fully automated or integrated [9]. Another research by Singh et al. proposed the automation of manual penetration testing using Bash shell scripts. This approach successfully reduced the time, effort, and potential for human error in stages such as reconnaissance, scanning, enumeration, exploitation, and documentation. The script implementation allowed for the automation of processes like domain information gathering, subdomain enumeration, availability validation, and screenshotting of active subdomains. Their results showed that automation via Bash shell scripts improved efficiency, expanded coverage, and produced more consistent findings compared to fully manual penetration testing [10]. One of the challenges in automated penetration testing is the blocking of IP addresses by websites. This occurs because automation can run multiple tools simultaneously, creating a burstiness pattern that is detected as suspicious activity [11]. To overcome this, proxychains technology can be utilized, where each request is distributed through multiple proxy servers, thus decentralizing IP address usage [12].

Related research has been conducted to develop technologies against the latest attacks. Tang et al. developed a web attack detection framework based on the OWASP Top 10 2021, covering Remote Code Execution (RCE), Injection, Local File Inclusion (LFI), Cross-Site Scripting (XSS), XML External Entity (XXE), Open Redirect, Carriage Return Line Feed (CRLF), and Deserialization Attacks [13]. Meanwhile, Albalawi et al. showed that SQL Injection and LFI accounted for 18% and 6% of web defacement cases, respectively, making them the most frequently exploited attack vectors [14]. Based on these findings, this research focuses on five primary vulnerability types: SQL Injection (SQLi), Cross-Site Scripting (XSS), Local File Inclusion (LFI), Open Redirect, and Carriage Return Line Feed (CRLF).

The Penetration Testing Execution Standard (PTES) provides detailed planning and a clear foundational structure for each phase of penetration testing, enabling systematic vulnerability identification and yielding easily understandable security recommendations [15]–[19]. Additionally, the OWASP WSTG version 4.2 offers a comprehensive guide that ensures all potential vulnerabilities are identified and evaluated [20]. Previous research has also shown that CAPEC can provide insights into common attack patterns, helping to design more effective mitigation strategies [21]. In recent MITRE documentation, the CAPEC v3.9 update (2023) provides a more comprehensive mapping of modern attack patterns and up-to-date mitigation guidance based on evolving cyber incident landscapes [22].

The explicit novelty of this research lies in the development of a fully integrated, Telegram-driven penetration testing framework that uniquely aligns standardized security testing protocols with automated execution. Unlike previous studies that utilize Telegram bots merely as remote command-

Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

line interfaces requiring technical proficiency, this study contributes a framework that automates the entire PTES and OWASP WSTG v4.2 workflows. Furthermore, the integration of automated proxychains rotation represents a significant technical advancement, ensuring scanning persistence against IP-based blocking mechanisms, a critical feature often overlooked in existing automated bot-based tools.

This research aims to implement this novel framework into an automated tool to assist local government agencies in conducting regular audits. To address IP address blocking by websites, the tool is developed using proxy chains, ensuring the IP address changes when running multiple tools concurrently. For testing purposes, a dummy website designed with known vulnerabilities is used. This tool is expected to provide a resource-efficient solution for detecting vulnerabilities such as SQLi, XSS, and LFI on government websites without requiring deep technical expertise.

2. METHOD

This study employed a mixed-methods approach, combining qualitative and quantitative techniques. The qualitative aspect involved interviews to explore user requirements, while the quantitative aspect consisted of system testing through Functional Testing, Performance Testing, and User Acceptance Testing (UAT). This approach was chosen to ensure that the research outcomes not only reflect practical needs but are also validated through measurable testing.

The system was developed using the Waterfall model of the System Development Life Cycle (SDLC), which includes the following stages: analysis, design, implementation, testing, and maintenance. In the analysis stage, the system and feature requirements that became the focus of the research were identified. The design stage utilized Unified Modeling Language (UML) diagrams to model user interactions and system process flows. Implementation was carried out by building a Telegram bot based on Python and Bash, connected to a Kali Linux server, and integrating the PTES framework to automate the penetration testing phases. Testing was conducted to ensure the system's functions and performance met the required standards, while the maintenance stage was aimed at resolving any issues that arose during testing.

2.1. Type of Research

2.1.1. Qualitative Approach

Qualitative research is a method that involves collecting and analyzing textual or narrative data to be understood and interpreted using interpretive methods [23]. The qualitative approach in this study was conducted through interviews with the relevant agency to gather information regarding user requirements, problems encountered during manual penetration testing, and expectations for the automated penetration testing tool to be developed.

2.1.2. Quantitative Approach

Quantitative research is a method that involves collecting and analyzing numerical data to answer specific questions, with the results expressed in the form of statistics [23]. The quantitative approach in this study was implemented by testing the tool through Functional Testing, Performance Testing, and User Acceptance Testing (UAT).

2.2. *System Development Life Cycle* (SDLC)

The development of the automated penetration testing tool based on a Telegram bot in this research utilized the System Development Life Cycle (SDLC) with the Waterfall model. This model describes a sequential software development process where each phase must be fully completed before proceeding to the next. The SDLC provides structure and control to ensure the development process is

Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

effective and efficient, making it suitable for projects where requirements are clearly defined from the outset and minimal changes are expected during development [24]. The phases of the SDLC are illustrated in Figure 1.

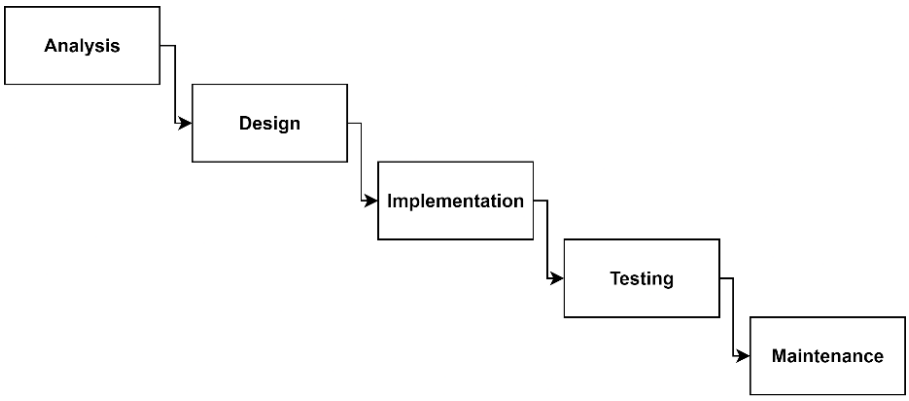


Figure 1. SDLC *Waterfall*

2.2.1. *Analysis*

This phase aims to identify the functional and non-functional requirements that form the basis for the development of the Telegram bot. This stage also serves as the pre-engagement phase within the PTES framework.

2.2.2. *Design*

This phase involved formulating the OWASP WSTG v4.2 indicators that would be applied. Furthermore, the system was designed using a flowchart to ensure the workflow was more easily understandable and structured. This flowchart illustrates the sequence of processes, starting from user authentication and feature selection, through to the execution of the automated penetration test and the generation of a report. Every decision branch and interaction between the user and the system is displayed, thereby facilitating the visualization of the overall system logic [25].

2.2.3. *Implementation*

The implementation phase was carried out by building a Telegram bot to serve as the interface for the automated penetration testing tool, using Python, the Telegram Bot API, and Bash scripts. The tool was designed to test for five types of vulnerabilities: CRLF Injection, Local File Inclusion (LFI), Cross-Site Scripting (XSS), SQL Injection (SQLi), and Open Redirect. This system integrates the PTES framework and the OWASP WSTG v4.2 indicators and is equipped with proxychains to prevent IP address blocking. Automation is executed via Bash scripts connected to the bot with pre-defined testing parameters, allowing users to initiate a scan simply by inputting a domain without needing to write manual commands. PTES (Penetration Testing Execution Standard) was used as the primary framework for conducting the penetration test, which encompasses the phases of information gathering, vulnerability analysis, exploitation, post-exploitation, and reporting. Each phase was automated using Bash scripts and controlled via the Telegram bot, ensuring that the testing process could be executed sequentially. The PTES Framework comprises the following stages: information gathering, vulnerability analysis, exploitation, post-exploitation, and reporting.

A. Information Gathering

The objective of this stage is to identify the website's architecture, assets, and potential vulnerabilities. The data gathered encompasses server details, open ports, subdomains, URL parameters, and the directory structure.

Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

B. Vulnerability Analysis

In this stage, a vulnerability analysis of the website is conducted using tools integrated within the Telegram bot. The primary information used is a URL with parameters that can be injected according to the specific type of vulnerability. The testing focuses on CRLF, SQL Injection, XSS, Local File Inclusion (LFI), and Open Redirect.

C. Exploitation

In this stage, the tool validates vulnerabilities against the website using tools that fulfill the vulnerability validation indicators of the OWASP WSTG v4.2.

D. Post-Exploitation

The post-exploitation stage is a continuation of the exploitation stage, focusing exclusively on processing vulnerabilities that have been successfully validated. Upon confirmation of a valid vulnerability, the post-exploitation module processes the name of the identified vulnerability, correlating it with pre-prepared CAPEC mitigation recommendations. The program then automatically generates remediation advice based on the corresponding CAPEC data.

E. Reporting

The reporting stage aims to present the results of the penetration testing process. The program displays processed output containing key information, which includes an executive summary, a list of validated vulnerabilities, and remediation recommendations. All these findings are then compiled into a report file that can be downloaded via the Telegram bot.

2.2.4. *Testing*

A. *Functional Testing*

Functional testing was conducted using the black box testing method to ensure that each feature of the Telegram bot operates according to its functional requirements. The test was performed by providing specific input from the user and verifying whether the generated output matched the expected results.

B. *Performance Testing*

Performance testing was conducted on the dummy website http://testphp.vulnweb.com. The objective was to assess the bot's capability in performing penetration tests against various vulnerabilities and to ensure that the results provided were both valid and consistent. Additionally, a manual penetration test was also performed in this phase to validate the findings of the vulnerabilities discovered by the automated penetration testing Telegram bot.

C. *User Acceptance Testing* (UAT)

This testing aimed to determine the level of user acceptance of the tool. The assessment was based on three software quality aspects according to ISO 9126, namely reliability, usability, and efficiency. Measurements were performed using a Likert scale, where a score of 1 indicated “strongly disagree” and a score of 5 indicated “strongly agree” [26].

3. RESULT

3.1. Analysis Results

The results of the analysis consist of the functional and non-functional requirements, which are detailed in Table 1. The functional and non-functional requirements serve as the foundation for the

Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

subsequent design and implementation phases. The implementation of these requirements is measured and validated through Functional Testing and User Acceptance Testing (UAT).

Table 1. Functional and Non-functional Requirements

No	Requirement Type	Description
1	Functional	The user can input the target subdomain for the penetration test. The user can select specific test types or tests for all five vulnerabilities simultaneously. The user must authenticate before using the Telegram bot. The user can interact with the system via an inline keyboard in the Telegram bot. The user can change the target subdomain for the penetration test. The user can cancel an ongoing penetration test process. The tool provides a menu with a quick guide for usage. The user can receive the test results in the form of a report file.
2	Nonfunctional	The tool shall present process logs in a structured and user-friendly format. The tool must have a simple and intuitive interface.

3.2. Design Results

A list of the WSTG indicators used in this research is presented in Table 2. Furthermore, the flowchart for the automated penetration testing tool is illustrated in Figure 2.

Table 2. Applied WSTG Indicators

No	Indicator Type	Indicator Code
1	*Information Gathering* (INFO)	WSTG-INFO-01, WSTG-INFO-02, WSTG-INFO-03, WSTG-INFO-04, WSTG-INFO-05, WSTG-INFO-06, WSTG-INFO-07, WSTG-INFO-08, WSTG-INFO-09, WSTG-INFO-10
2	*Input Validation Testing* (INPV)	WSTG-INPV-01, WSTG-INPV-05, WSTG-INPV-11, WSTG-INPV-15
3	*Client-Side Testing (CLNT)*	WSTG-CLNT-04

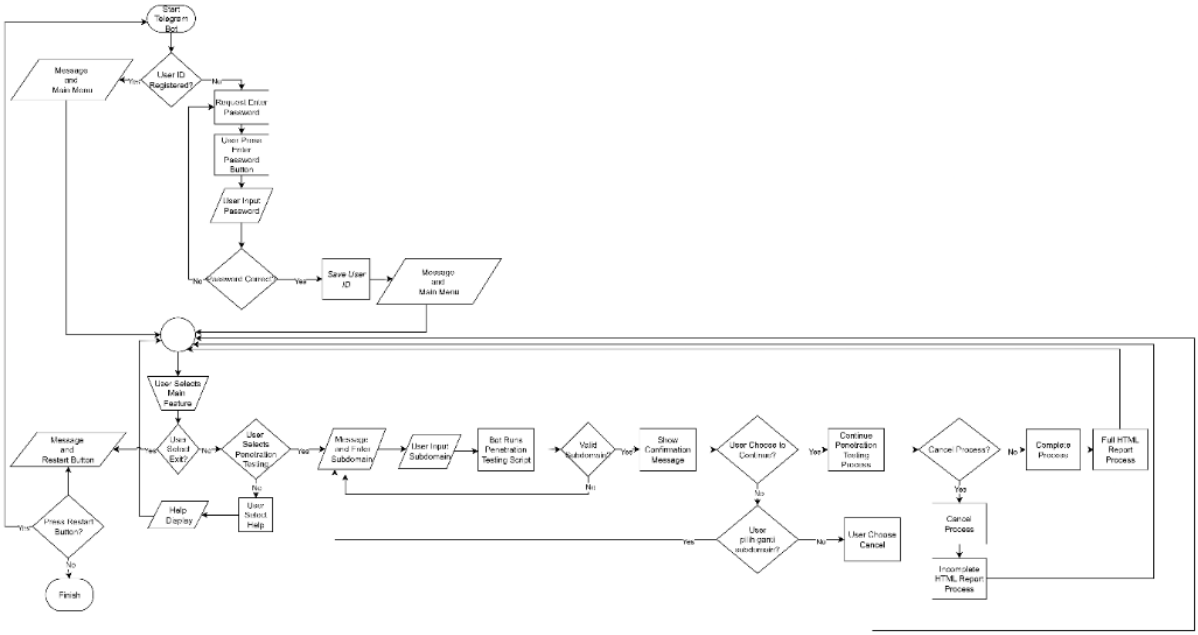


Figure 2. Automated Penetration Testing Tool Flowchart

Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

3.3. Implementation Results

The automated penetration testing Telegram bot was developed using the Python programming language, utilizing the Telegram Bot API as its interface and Bash scripts to execute penetration testing tool commands on a Kali Linux server. The tool is integrated with the Penetration Testing Execution Standard (PTES) framework and the OWASP Web Security Testing Guide (WSTG) v4.2 indicators, ensuring that the testing phases proceed in a structured manner, from information gathering, vulnerability analysis, and exploitation, to post-exploitation and reporting.

3.3.1. Authentication Feature Interface

The bot's initial interaction requires the user to perform authentication, a process depicted in Figure 3. The bot displays a welcome message along with security instructions and then provides an 'Enter Password' button. Once the correct password is provided, the user's ID is saved, ensuring that authentication is not required for subsequent sessions.



Figure 3. Authentication Feature

3.3.2. Penetration Testing Feature Interface

The user can select the type of vulnerability to be tested. Subsequently, the user is prompted to enter the target subdomain, as shown in Figure 4. After the target is entered, the bot initiates the penetration testing process and displays the execution logs in real-time. Additionally, the bot provides a “Cancel Process” button to stop the test if needed, ensuring the user maintains full control throughout the process.



Figure 4. Penetration Testing Feature

Jurnal Teknik Informatika (JUTIF)
P-ISSN: 2723-3863
E-ISSN: 2723-3871
Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

3.3.3. Guide Feature Interface

The bot is equipped with a guide menu to help users understand its functions and the available test types. As shown in Figure 5, the bot displays a list of the vulnerabilities that can be tested, including CRLF Injection, Local File Inclusion (LFI), Open Redirect, SQL Injection, and Cross-Site Scripting (XSS), as well as the Information Gathering stage.

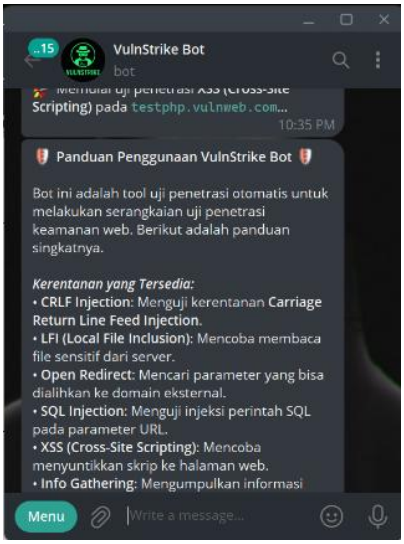


Figure 5. Guide Feature

3.3.4. Report Output Sample

The automated reporting module represents the final critical phase of the Penetration Testing Execution Standard (PTES) workflow implemented within the system. After completing all predefined scanning modules, the Bash script consolidates the findings into a downloadable document, which is then delivered automatically to the user via the Telegram Bot API. Figure 6 illustrates a sample of the raw output for an identified SQL Injection vulnerability on a test target (e.g., dummy.web.id). The report explicitly detailed the testing parameters, including the request type (POST) and the exact payload that successfully triggered the vulnerability. This raw output serves as technical evidence to validate the existence of the security flaw. In the final report document provided to users, this data is structured alongside standardized mitigation guidance to assist administrators in efficient remediation.

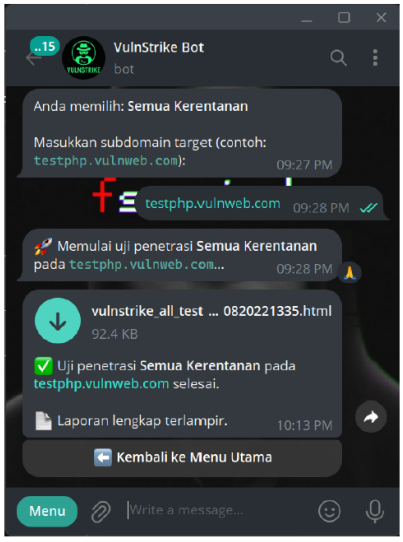


Figure 6. Output Sample of the SQL Injection Vulnerability Detection

Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

3.3.5. Quit Feature Interface

In the main menu, the bot displays a quit option. When selected, it presents a message as shown in Figure 7. The message consists of a “Thank You” note and a "Start Test Again" option for users who wish to conduct another test.



Figure 7. Quit Feature

3.4. Testing Results

Following the successful design and implementation of the tool, the research continued with a series of evaluations, including functional, performance, and User Acceptance Testing (UAT).

3.4.1. Functional Testing Results

Functional testing was conducted using eleven test scenarios, covering authentication for new and registered users, subdomain input validation, vulnerability test execution, process cancellation, the help feature, as well as exiting and restarting the system. The test results, presented in Table 3, indicate that all scenarios passed, thus confirming that the Telegram bot functions in accordance with the predefined functional requirements.

Table 3. Functional Testing Results

Scenario ID	Scenario Name	Result
FT-01	New User Authentication - Success	Passed
FT-02	New User Authentication - Failure	Passed
FT-03	Registered User Authentication	Passed
FT-04	Running a Single Module Test	Passed
FT-05	Invalid Subdomain Input	Passed
FT-06	Process Cancellation	Passed
FT-07	Test All Vulnerabilities	Passed
FT-08	Help Feature	Passed
FT-09	Quit and Restart	Passed
FT-10	Verify Report with Findings	Passed
FT-11	Verify Report with No Findings	Passed

3.4.2. Performance Testing Results

Performance testing was carried out on a dummy site to assess the Telegram bot's capability in performing vulnerability penetration tests. The results, presented in Table 4, show a total of 186 findings.

Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

The breakdown includes: 169 Cross-Site Scripting (XSS) findings that triggered JavaScript execution and displayed a pop-up; 10 SQL Injection findings that indicated successful query injection into the database; 6 Open Redirect findings resulting in redirection to an external domain; and 1 Local File Inclusion (LFI) finding that successfully accessed a server file. Meanwhile, no vulnerabilities were discovered during the CRLF Injection test. All vulnerabilities were subsequently re-tested manually, which yielded identical results, including the confirmation that no CRLF vulnerabilities were found.

Table 4. Performance Testing Results

Vulnerability Name	Number of Vulnerable URLs	Test Result
CRLF *Injection*	-	No header injection detected.
Local File Inclusion (LFI)	1	Server file successfully loaded/accessed.
Open Redirect	6	Redirection to an external domain detected,
Cross-Site Scripting (XSS)	169	JavaScript execution detected, displaying a pop-up.
SQL *Injection*	10	Database information retrieved via SQL query injection.
Total Temuan	186	

3.4.3. User Acceptance Testing (UAT) Results

The UAT results indicate an excellent level of user acceptance, with an average score of 95.75%. The aspects of reliability and usability each received a score of 96%, while the efficiency aspect achieved 95.2%. These results demonstrate that the Telegram bot is stable, easy to use, and efficient in generating reports, thereby meeting the user acceptance criteria. The detailed UAT results are presented in Table 5.

Table 5. UAT Results

Characteristic	Actual Score	Ideal Score	Acceptance Percentage	Criteria
Realibility	96	100	96	Excellent
Usability	168	175	96	Excellent
Efficiency	119	125	95,2	Excellent
Total	**383**	**400**	**95,75**	**Excellent**

4. DISCUSSIONS

The research results indicate that the developed automated penetration testing Telegram bot functions in accordance with user requirements. All scenarios in the functional testing were successfully met, which means the system can perform authentication, input validation, vulnerability testing, process cancellation, and report presentation as designed. These findings affirm that the bot provides a practical solution for users without deep technical expertise. In the performance testing, the system successfully detected 186 vulnerabilities, with most findings being XSS and SQL Injection, followed by Open Redirect and LFI, while no CRLF vulnerabilities were detected. These results were corroborated by manual re-testing, which yielded the same outcomes. Furthermore, the UAT results showed a user acceptance rate of 95.75%, categorized as "excellent."

In comparison to the research by Hafiz et al., which developed a Telegram bot to run several tools but still demanded technical understanding from the user, this research presents a fully automated system that is easier to operate. Meanwhile, the research by Singh et al. demonstrated the effectiveness of Bash script-based automation but required an additional Linux server, whereas this tool operates with direct integration through the Telegram bot. Thus, this research offers advantages in its integration of Bash scripts, the Telegram Bot API, PTES, and OWASP WSTG v4.2, as well as the use of proxychains to

Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

overcome IP blocking, thereby making a concrete contribution to improving the efficiency of penetration testing.

However, several limitations in this study must be acknowledged; the performance evaluation was conducted exclusively on a controlled dummy website with pre-defined vulnerabilities, meaning its effectiveness against complex, real-world security infrastructures and modern Web Application Firewalls (WAF) requires further validation. Additionally, the current framework is strictly limited to black-box testing methodologies, focusing on external scanning without access to the target's internal source code or system architecture, which precludes the depth of analysis found in grey-box or white-box testing. Lastly, while the tool aligns with PTES and OWASP WSTG v4.2 standards, it does not yet encompass the full spectrum of the OWASP Top 10 vulnerabilities, as it currently concentrates on five major attack vectors: SQLi, XSS, LFI, Open Redirect, and CRLF.

5. CONCLUSION

This research successfully developed an automated penetration testing Telegram bot based on PTES and OWASP WSTG v4.2, capable of executing integrated penetration tests. The test results prove that the system meets all functional requirements, detects four out of the five primary vulnerabilities tested, and complies with the OWASP WSTG v4.2 indicators. Furthermore, the user acceptance rate in the User Acceptance Testing (UAT) reached an average of 95.75%, categorized as "excellent," which reflects that the tool is stable, easy to use, and efficient. Overall, this system can be considered a viable alternative for conducting penetration tests, particularly for institutions that require a practical and resource-efficient method. The contribution of this research lies in providing a penetration testing tool that is automated, standardized, and user-friendly. Future research could be developed by expanding the scope of vulnerabilities and employing grey box testing methods to further support cybersecurity protection in the government administration sector.

REFERENCES

[1] F. A. Putra, "Tata Kelola Ekosistem Berbagi Informasi Keamanan Siber pada Information Sharing and Analysis Center (ISAC) Sektor Pemerintah Daerah di Indonesia," *Info Kripto*, vol. 16, no. 1, pp. 23–32, 2022, doi: 10.56706/ik.v16i1.39.

[2] K. P. A. N. dan R. B. (PANRB), "Kementerian PANRB Umumkan Hasil Evaluasi SPBE Tahun 2023," 2024. https://www.menpan.go.id/site/berita-terkini/kementerian-panrb-umumkan-hasil-evaluasi-spbe-tahun-2023.

[3] A. Y. I. Mauliza, R. D. S. Machmudi, and R. Indrarini, "Pengaruh Perlindungan Data Dan Cyber Security," *J. Ilm. Bid. Sos. Budaya , Teknol. dan Pendidik.*, vol. 1, no. 11, pp. 2497–2516, 2022, [Online]. Available: https://publish.ojs-indonesia.com/index.php/SIBATIK/article/view/395/345.

[4] Badan Siber dan Sandi Negara, "Lanskap Keamanan Siber Indonesia 2024," 2025. [Online]. Available: https://www.bssn.go.id/wp-content/uploads/2025/02/LANSKAP-KEAMANAN-SIBER-2024-1.pdf.

[5] N. F. Juhara, M. Amalia, and A. Mulyana, "Efektivitas Penegakan Hukum terhadap Judi Online di Indonesia : Analisis Yuridis dan Sosiologis," *J. Contemp. Law Stud.*, vol. 2, no. 2, pp. 153–164, 2025, [Online]. Available: https://journal.pubmedia.id/index.php/lawstudies/article/view/3353/3466.

[6] L. M. Zagi, G. P. Digdo, and W. Shalannanda, "Just Dork and Crawl: Measuring Illegal Online Gambling Defacement in Indonesian Websites," *arXiv Prepr.*, 2025, [Online]. Available: https://arxiv.org/abs/2508.19368.

[7] Badan Siber dan Sandi Negara, *Peraturan Badan Siber dan Sandi Negara Nomor 4 Tahun 2021 tentang Pedoman Manajemen Keamanan Informasi Sistem Pemerintahan Berbasis Elektronik dan Standar Teknis dan Prosedur Keamanan Sistem Pemerintahan Berbasis Elektronik*. 2021.

[8] B. A. Bin Arfaj, S. Mishra, and M. Alshehri, "Efficacy of Unconventional Penetration Testing

Vol. 7, No. 5, October 2026, Page. 4756-4767
https://jutif.if.unsoed.ac.id
DOI: https://doi.org/10.52436/1.jutif.2026.7.5.5738

Practices," *Intell. Autom. Soft Comput.*, vol. 31, no. 1, pp. 223–239, 2022, doi: 10.32604/IASC.2022.019485.

[9] M. H. Andifaisa, D. Pramono, and B. T. Hanggara, "Pembangunan Sistem Informasi Manajemen Toko Menggunakan Telegram Bot Berbasis Web (Studi Kasus Noor Electric)," *J. Pengemb. Teknol. Inf. dan Ilmu Komput.*, vol. 5, no. 9, pp. 3662–3669, 2021, [Online]. Available: https://j-ptiik.ub.ac.id/index.php/j-ptiik/article/view/9667/4351.

[10] A. K. Singh, G. Kumar, and P. K. Vishwakarma, "Automation in Manual Penetration Testing Using Bash Shell Script," *Int. Res. J. Mod. Eng. Technol. Sci.*, 2023, doi: 10.56726/irjmets40392.

[11] D. Jeon and B. Tak, "BlackEye: automatic IP blacklisting using machine learning from security logs," *Wirel. Networks*, vol. 28, no. 2, pp. 937–948, 2022, doi: 10.1007/s11276-019-02201-5.

[12] E. Blancaflor, B. J. Ordonez, D. I. Pajutrao, and K. G. Verano, "Enhancing Online Privacy and Security through Proxy Chaining in Kali Linux: A Comprehensive Analysis," pp. 51–55, 2024, doi: 10.1145/3675585.3675593.

[13] S. Ramezany, R. Setthawong, and T. Tanprasert, "A Machine Learning-based Malicious Payload Detection and Classification Framework for New Web Attacks," *19th Int. Conf. Electr. Eng. Comput. Telecommun. Inf. Technol. ECTI-CON 2022*, pp. 1–4, 2022, doi: 10.1109/ECTI-CON54298.2022.9795455.

[14] M. Albalawi, R. Aloufi, N. Alamrani, N. Albalawi, A. Aljaedi, and A. R. Alharbi, "Website Defacement Detection and Monitoring Methods: A Review," *Electron.*, vol. 11, no. 21, 2022, doi: 10.3390/electronics11213573.

[15] D. Dalalana Bertoglio and A. F. Zorzo, "Overview and open issues on penetration test," *J. Brazilian Comput. Soc.*, vol. 23, no. 1, pp. 1–16, 2017, doi: 10.1186/s13173-017-0051-1.

[16] S. U. Sunaringtyas and D. S. Prayoga, "Implementasi Penetration Testing Execution Standard Untuk Uji Penetrasi Pada Layanan Single Sign-On," *Edu Komputika J.*, vol. 8, no. 1, pp. 48–56, 2021, doi: 10.15294/edukomputika.v8i1.47179.

[17] M. F. Safitra, M. Lubis, and A. Widjajarto, "Security Vulnerability Analysis using Penetration Testing Execution Standard (PTES): Case Study of Government's Website," *ACM Int. Conf. Proceeding Ser.*, no. 3, pp. 139–145, 2023, doi: 10.1145/3592307.3592329.

[18] F. P. Utama and R. M. H. Nurhadi, "Uncovering the Risk of Academic Information System Vulnerability through PTES and OWASP Method," *CommIT J.*, vol. 18, no. 1, pp. 39–51, 2024, doi: 10.21512/commit.v18i1.9384.

[19] I. D. G. G. Dharmawangsa, G. M. A. Sasmita, and I. P. A. E. Pratama, "Penetration Testing Berbasis OWASP Testing Guide Versi 4.2 (Studi Kasus: X Website)," *JITTER J. Ilm. Teknol. dan Komput.*, vol. 4, no. 1, p. 1613, 2023, doi: 10.24843/jtrti.2023.v04.i01.p06.

[20] I. D. G. G. Dharmawangsa, G. M. A. Sasmita, and I. P. A. E. Pratama, "Penetration Testing Berbasis OWASP Testing Guide Versi 4.2 (Studi Kasus: X Website)," *JITTER J. Ilm. Teknol. dan Komput.*, vol. 4, no. 1, p. 1613, 2023, doi: 10.24843/jtrti.2023.v04.i01.p06.

[21] B. S. Meyers, S. F. Almassari, B. N. Keller, and A. Meneely, "Examining Penetration Tester Behavior in the Collegiate Penetration Testing Competition," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 3, 2022, doi: 10.1145/3514040.

[22] CAPEC, "Common Attack Pattern Enumeration and Clasification - CAPEC™: A Community Knowledge Resource for Building Secure Software," 2023.

[23] H. Taherdoost, "What are Different Research Approaches ? Comprehensive Review of Qualitative , Quantitative , and Mixed Method Research , Their Applications , Types , and Limitations," . *J. Manag. Sci. Eng. Res.*, vol. 05, no. 01, pp. 53–63, 2022, doi: 10.30564/jmser.v5i1.453.

[24] A. Saravanos and M. X. Curinga, "Simulating the Software Development Lifecycle: The Waterfall Model," *Appl. Syst. Innov.*, vol. 6, no. 6, 2023, doi: 10.3390/asi6060108.

[25] Zainab Tuasamu *et al.*, "Analisis Sistem Informasi Akuntansi Siklus Pendapatan Menggunakan DFD dan Flowchart Pada Bisnis Porobico," *J. Bisnis dan Manaj.*, vol. 1, no. 2, pp. 495–510, 2023, doi: 10.61930/jurbisman.v1i2.181.

[26] H. Hasugian, Wulandari, and Nofiyani, "User Acceptance Testing (UAT) Pada Electronic Data Preprocessing Guna Mengetahui Kualitas Sistem," *J. Mhs. Ilmu Komput.*, vol. 4, no. 1, pp. 20–27, 2023, doi: https://doi.org/10.24127/ilmukomputer.v4i1.3383.